Enhance an existing app using test driven development

20 hours Medium License

Last updated on 4/27/23

Understand the Red Green Refactor Methodology

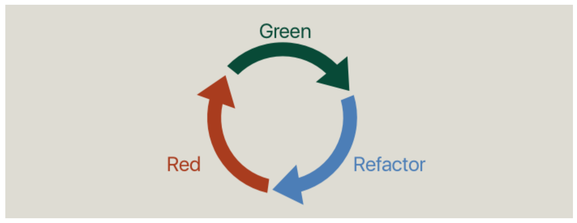
Created by

OpenClassrooms, Leading E-Learning Platform

Now that we're familiar with the concept of Test Driven Development, in order to execute it properly we need to learn its key methodology: "**Red, Green, Refactor.**"

Introducing the method

This method consists of 3 phases that are repeated continuously: *Red*, *Green,* and *Refactor*.



Red Green Refactor scheme

Let's look at them more closely:

Red

The first phase is writing a test. Since at the moment of writing, the corresponding code that supports the tested functionality does not exist, the test is destined to fail. 💔 Hence, it gets marked RED.

Green

The second phase is writing the production code - the intended functionality that will correspond to the test. Here, less attention is paid to the elegance of the solution. The primary focus at this moment is to pass the test and turn it GREEN! 💚

Refactor

The next phase is to refactor both the code and the tests. The goal here is to pay attention to the contents and find ways to make improvements.

Some questions to ask:

- Is the code following a proper style?
- Are there repetitions?
- Does class have a single responsibility?
- Does the method name reflect its purpose?
- ...etc.

This phase is an opportunity to improve the quality of your code! 💪

Small improvements make a big difference

Following this method, you'll go through a cycle. As soon as you reach the last phase, you return to the first one. The cycle must be very short - about 5 minutes!

> How can we code anything useful in 5 minutes? 😉

It may be hard to believe at first, but you *can* code very small sets of instructions in this amount of time to get started, and then improve and expand during the next iterations. For example, in the beginning, there could be a cycle that implements a test to verify that a class simply exists.

The idea is to write a test that requires as little supporting code as possible!

> Isn't it overkill to write a test just to verify a class declaration? 😶

Sure, it sounds a little weird at first but soon proves beneficial. The advantage of advancing in small increments is that the risk of error is almost non-existent and you are always covered.

It's like climbing a mountain and every 5 minutes you put a new safely net beneath you. You don't risk falling far! It's much better than moving the net every 2 hours!

> Do 5 minute tests result in too many of them? 😟

Not exactly - because of the last phase: refactor. This phase is also meant to refactor tests; those that become obsolete can be altered or removed.

As you keep cycling through this process, your test suite is always live - it evolves continuously with the project and reflects exactly what the production code is about at every single moment!

Advantages

I'd like to highlight two major advantages this method presents:

1. You are not perfect!

The first one enables you to gradually perfect your work in small increments.

Let's face it, in most cases, the fist time we write a piece of code, it won't be perfect! Even professional musicians never play a new piece perfectly for the first time. 🎷 Writing code is the same!

Instead of writing a big chunk of code with a promise to improve it later, you are constantly presented with numerous opportunities to improve! This dramatically increases your chances of producing code of outstanding quality!

> 6 minute cycles are also fine, by the way! 😉

2. Refactor in the green

And the second HUGE advantage is that you *only* refactor in the GREEN - after the test has passed and the function is working! Any possibility of breaking functionality while implementing "clever" solutions will impact the test result - it will turn red, which will keep you alert at all times and force you to make sure all improvements are still valid!

This method encourages you to refactor only from green to green!

Let's Recap!

- The Red, Green, Refactor method consists of three phases:
  - *Red* - write a test that **fails.**
  - *Green* - implement the test-supporting functionality to **pass the test.**
  - *Refactor* - **improve** the production code AND the tests to absolute perfection.
- The Red, Green, Refactor cycle should be as *short* as possible.
- The main objective of this method is to advance in small increments.

Discover Test Driven Development (TDD) Add a feature in TDD

Teacher

Olga Volkova

Fascinated by limitless opportunities in the universe of unknown. iOS engineer, interaction designer, entrepreneur, educator, writer.